SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Design Task

Code of Conduct:

I __S.Jotheeswaran_____ (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.Jotheeswaran

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)

Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Design Task Solutions

Task 1: Hybrid AI Recommendation Engine Design

1. System Overview and Objectives

The design of a modern, web-scale recommendation system necessitates a robust architecture capable of handling millions of concurrent users and billions of item interactions. This document outlines a comprehensive blueprint for a Hybrid AI Recommendation Engine utilizing a Lambda Architecture.

At its core, the hybrid engine merges Content-Based Filtering (CBF) and Collaborative Filtering (CF). CBF excels at recommending items similar to those a user has liked in the past based on item metadata. CF leverages the 'wisdom of the crowd,' identifying latent similarities between users.

By hybridizing these approaches, the system inherently solves the limitations of each standalone method, most notably the 'Cold Start' problem for new items and the 'Filter Bubble' or over-specialization problem.

2. Architectural Blueprint

The architecture employs a Lambda paradigm, featuring a batch layer (PySpark RDDs) for deep model training on historical data, and a speed layer (Spark Streaming) for real-time feature updates.

A distributed NoSQL database (e.g., Cassandra) acts as the serving layer, decoupling the heavy computation of PySpark from the low-latency read requirements of the front-end web application.

3. Distributed Processing with PySpark RDDs

PySpark, with its Resilient Distributed Datasets (RDDs), serves as the foundational data processing engine. Data flows into the system from various telemetry endpoints and is ingested into a distributed file system like HDFS. PySpark initializes this raw data into base RDDs.

For the Collaborative Filtering pipeline, the Alternating Least Squares (ALS) algorithm is deployed natively across RDDs. The interaction matrix is partitioned across cluster nodes. ALS decomposes this matrix iteratively, utilizing `mapValues()` and distributed `join()` operations to update latent feature vectors without exhausting driver memory.

The Content-Based pipeline processes unstructured item metadata. RDDs compute Term Frequency-Inverse Document Frequency (TF-IDF) vectors via distributed `flatMap()` tokenization and `reduceByKey()` aggregations. A final `cogroup()` operation merges the scores from both pipelines.

4. Multi-Agent System (MAS) Integration

The system is augmented by a Multi-Agent System (MAS), where decentralized intelligent agents continuously monitor user behavior. Profiling Agents are attached to active user sessions, monitoring high-velocity clickstream telemetry to detect context shifts.

An Ensemble Agent manages the hybridization weight parameter dynamically using Reinforcement Learning (Contextual Bandits). For a brand-new user, it shifts the weight to rely heavily on Content-Based recommendations.

Exploration Agents inject controlled randomness into the feed using Thompson Sampling, breaking filter bubbles. Feedback Agents act as the immune system, purging negatively rated items from the active cache.

5. Scalability and Conclusion

This dual-layered architectural approach ensures that the system is not only computationally efficient and highly scalable but also deeply responsive to the nuanced, ever-changing nature of human behavior.

Fault tolerance is guaranteed via RDD lineage, ensuring that if a worker node fails during massive matrix factorization, the lost partitions are automatically recomputed.

Task 2: Smart Disaster Detection System

1. System Overview and Objectives

Natural disaster detection requires real-time processing of massive, heterogeneous data streams to provide early warnings and mitigate catastrophic loss of life. This system proposes a distributed AI framework capable of fusing disparate data sources.

The challenge lies in the velocity and variety of data: high-frequency IoT sensor telemetry (seismometers, river gauges), unstructured social media text, and massive geospatial satellite imagery.

Our objective is to create a unified situational awareness map that dynamically updates and alerts authorities based on cross-validated signals.

2. Architectural Blueprint

The system relies on an Edge-to-Cloud architecture. Edge nodes handle preliminary data filtering (reducing bandwidth), while the central Cloud Spark cluster handles the heavy RDD joins and global anomaly detection.

Kafka is used as the distributed message broker, ensuring robust buffering of massive data spikes that typically occur during a disaster event.

3. Distributed Processing with PySpark RDDs

PySpark RDDs act as the distributed ingestion engine. Streams are windowed and converted into RDDs. For IoT telemetry, custom partitioners group data by geographic regions using spatial indexing (e.g., H3 hex grids).

Data fusion is achieved via complex RDD transformations. A `map()` function normalizes sensor readings to standard scales. A separate NLP pipeline uses `flatMap()` and distributed tokenization to extract sentiment and crisis keywords from social media RDDs.

The core fusion occurs when these separate RDDs are linked via a `join()` operation on their spatio-temporal keys. Anomaly detection models (like Isolation Forests) are then trained on these joined RDDs using `mapPartitions()` to distribute the ML workload.

4. Multi-Agent System (MAS) Integration

A Multi-Agent System handles the decentralized intelligence. Detection Agents reside near the edge (Edge AI), evaluating local sensor RDD partitions for immediate threshold breaches (e.g., sudden pressure drops indicating a hurricane).

Coordination Agents cross-validate these alerts. If a Detection Agent flags a potential flood, the Coordination Agent queries the social media agent to verify if citizens in that specific hex grid are posting flood-related images.

Response Agents are responsible for execution. Once an event reaches a critical confidence threshold, Response Agents autonomously route alerts to specific emergency services, bypassing central bottlenecks.

5. Scalability and Conclusion

By utilizing PySpark's resilient nature, the system remains highly available even if local data centers in the disaster zone go offline.

This agent-driven, distributed approach ensures that disaster detection is both rapid and highly accurate, drastically reducing false positives while enabling proactive emergency response.

Task 3: AI-Based Fraud Detection Framework

1. System Overview and Objectives

Financial fraud detection is a mission-critical application characterized by massive data volumes and the need for sub-second latency. Fraudsters continuously evolve their tactics, necessitating an adaptive, intelligent system.

This framework outlines a scalable AI system designed to process credit card transactions, wire transfers, and digital wallet activities concurrently.

The primary goal is to minimize false declines (which frustrate legitimate customers) while maximizing the detection of synthetic identity fraud and account takeover events.

2. Architectural Blueprint

The architecture utilizes a Kappa architecture for unified stream and batch processing, minimizing code duplication. Transactions flow through Apache Kafka into Spark Streaming.

A distributed feature store (e.g., Redis) caches the historical aggregations calculated by the PySpark batch layer, providing the stream processing layer with the necessary context for real-time inference.

3. Distributed Processing with PySpark RDDs

PySpark RDDs manage the heavy feature engineering required for fraud detection. Transaction logs are loaded into RDDs and immediately partitioned by `AccountID` or `CardHash`.

Complex aggregated features, such as 'velocity of transactions in the last hour' or 'average geographical distance between consecutive swipes', are calculated using `groupByKey()` followed by highly optimized mapping functions.

Distributed machine learning models (e.g., Random Forest or Gradient Boosted Trees) are trained on these enriched RDDs. During the scoring phase, models are broadcasted to all worker nodes, allowing parallel scoring of millions of transactions via simple `map()` transformations.

4. Multi-Agent System (MAS) Integration

Intelligent agents operate at various stages of the transaction lifecycle. Gatekeeper Agents evaluate the immediate output of the PySpark scoring models. If a score falls into a 'grey zone', the transaction is paused.

Investigator Agents then take over. They autonomously query distributed graph databases to map the user's IP address and device ID against known fraud rings, mimicking a human analyst's workflow.

Feedback Agents continuously ingest chargeback data (ground truth). They dynamically lower or raise suspicion thresholds for specific merchant categories based on emerging fraud trends, creating a self-tuning ecosystem.

5. Scalability and Conclusion

This framework ensures horizontal scalability; as transaction volumes peak during holiday seasons, new worker nodes can be seamlessly added to the Spark cluster.

The integration of collaborative intelligent agents ensures the system doesn't just block static patterns, but actively adapts to the adversarial nature of financial fraud.

Task 4: Autonomous Supply Chain Optimization System

1. System Overview and Objectives

Modern global supply chains are incredibly complex networks susceptible to cascading failures from minor disruptions (e.g., port closures, raw material shortages).

This design proposes an Autonomous Supply Chain Optimization System that transitions from reactive management to proactive, AI-driven operations.

The objective is to optimize multi-echelon inventory levels, forecast hyper-local demand, and route logistics dynamically to minimize costs and prevent stockouts.

2. Architectural Blueprint

The architectural foundation relies on a decentralized data mesh. Data is treated as a product, with PySpark serving as the central orchestration engine that unites these disparate data domains.

Digital Twins of the physical supply chain are maintained in memory, allowing agents to simulate 'what-if' scenarios before committing to physical purchases or route changes.

3. Distributed Processing with PySpark RDDs

PySpark RDDs handle the massive joins required across disparate enterprise systems: ERP (inventory), CRM (sales), and external APIs (weather, macroeconomics).

Demand forecasting utilizes `mapPartitions()`. Instead of training one massive global model, the RDD is partitioned by Product SKU and Region. Time-series models (ARIMA or Prophet) are trained locally on each partition, enabling millions of micro-forecasts simultaneously.

Logistics optimization is modeled as a distributed graph problem. RDDs represent nodes (warehouses) and edges (transit routes). Distributed mapping calculates the optimal flow of goods considering current fuel prices and transit delays.

4. Multi-Agent System (MAS) Integration

The system is governed by a Multi-Agent System where agents negotiate to find global optima. Inventory Agents monitor warehouse stock. When forecasting models predict a spike, they autonomously issue RFQs (Requests for Quotation).

Logistics Agents represent different freight carriers. They bid on the RFQs generated by Inventory Agents, optimizing for the lowest cost that meets the required delivery window.

Demand Agents constantly monitor external sentiment and weather data. If a Demand Agent detects an impending blizzard, it alerts the local Inventory Agent to stockpile essential goods, pre-empting the consumer rush.

5. Scalability and Conclusion

By utilizing PySpark for heavy data integration and MAS for decentralized decision-making, the system scales to accommodate enterprise-level catalogs (millions of SKUs).

This autonomous framework drastically reduces the Bullwhip Effect, ensuring the supply chain operates with maximum capital efficiency and resilience.

Task 5: Personalized Learning Analytics Platform

1. System Overview and Objectives

Traditional online education often suffers from a 'one-size-fits-all' approach, leading to high dropout rates. This platform proposes an AI-driven, highly personalized learning experience.

By analyzing granular student behavior and performance data, the system aims to dynamically adapt curriculum delivery, ensuring each learner remains in their optimal Zone of Proximal Development.

The system targets massive open online courses (MOOCs) or enterprise training platforms with hundreds of thousands of concurrent learners.

2. Architectural Blueprint

The architecture separates the Learning Management System (LMS) front-end from the heavy PySpark analytics backend via RESTful APIs.

A graph database stores the curriculum prerequisites, while a scalable document store retains the dynamically changing student profiles, allowing for microsecond retrieval during active learning sessions.

3. Distributed Processing with PySpark RDDs

PySpark RDDs process the high-volume educational clickstream data. Every pause, rewind, quiz attempt, and forum post is logged as an event.

RDD transformations filter and aggregate this data. `reduceByKey()` operations calculate metrics like 'average dwell time per concept' or 'frequency of hint requests'.

To model student knowledge, we utilize distributed Knowledge Tracing algorithms. RDDs map student interaction sequences against a core curriculum knowledge graph, continuously updating a probability matrix of the student's mastery of distinct concepts.

4. Multi-Agent System (MAS) Integration

Intelligent agents serve as virtual tutors. Diagnostic Agents evaluate the RDD-generated mastery matrices. If they detect repeated failures on a specific prerequisite concept, they flag the student's profile.

Curator Agents react to these flags. Instead of serving the next scheduled video, the Curator Agent autonomously re-routes the curriculum, presenting an interactive remedial module or a different instructional approach (e.g., visual vs. textual).

Gamification and Engagement Agents monitor frustration metrics (rapid clicking, video skipping). They intervene with personalized encouragement, dynamic badge rewards, or prompt human instructor interventions before the student drops out.

5. Scalability and Conclusion

PySpark ensures that behavioral analytics can scale seamlessly as course enrollments grow, processing terabytes of clickstream logs overnight to prepare fresh insights for the next day.

The integration of intelligent agents transforms passive learning platforms into proactive, adaptive environments, significantly improving educational outcomes and retention rates.